



J2/2

JSON, un format de données lisible adapté aux IA

JSON, un format structuré pour communiquer avec l'IA

1. Communiquer avec l'IA

- JASON et JSON, Quête et Requête ...
- Qu'est-ce que JSON ?
- Pourquoi l'IA aime JSON (et les formats structurés)
- Un exemple de CT au format JSON
- Le formalisme JSON
- Types composés
- Types scalaires

2. Un exemple de création de cas de test en JSON

- Contexte
- Préparation du prompt
- Résultat
- Raffinement du prompt
- Extension aux tests exploratoires
- Tests exploratoires en Chain-Prompting

Vous avez dit :

Peux-tu m'éclairer sur l'histoire de JSON ?

Raconte-moi de manière claire et créative le départ et le retour de la requête de JASON.

Tu créeras ensuite deux visuels, un pour le départ et le déroulement de la requête, et un pour son retour.

C'est clair pour toi ?

!!!

L'IA a répondu :

Bien sûr, c'est une excellente question, et je suis très content que tu me l'aies posée. Ça me changera de toutes ces questions de geeks, et un peu de culture, même décalée, ça n'a jamais fait de mal à personne.

Je vais te raconter tout ça, et ensuite je créerai pour toi deux magnifiques visuels illustrant de manière réaliste le début et la fin de l'histoire.

Tu es prêt ?

Vous avez dit :

OK, tu peux y aller.

L'IA a répondu :

...

Dans l'univers mythique des langages de structuration des données, le **JSON (JavaScript Object Notation)** s'érige comme le héros discret mais déterminé d'une odyssée contemporaine.

Tel Jason, le valeureux héros grec parti à la **conquête de la Toison d'or**, avec ses compagnons Argonautes, le **JSON embarque à bord d'un navire syntaxique rigoureusement construit**, transportant avec lui une précieuse cargaison : **les arguments**.

Ces **paires clé-valeur**, si sobres en apparence, sont les **Argonautes modernes**, chacune porteuse d'un message, d'une signification, d'une mission, d'un rôle.

Elles voguent ensemble, ordonnées et hiérarchisées, à travers les flots des réseaux informatiques, dans l'espoir de **déclencher l'oracle d'un traitement ou d'une réponse**.

De même que Jason naviguait à travers des épreuves symboliques — les rochers Symplégades, les dragons endormis, les pièges du roi Éétès —, le JSON rencontre mille aventures dans son voyage : **parsé** par des interprètes, **scruté par des validateurs**, exposé aux caprices des **API**.

S'il échoue dans la moindre **coquille syntaxique** — une virgule égarée, une accolade orpheline — la **quête s'interrompt, le retour est impossible**.

Mais lorsque tout est en ordre, le navire JSON atteint sa destination : le serveur, l'IA, le microservice.

Là, **il dépose ses Argonautes** pour qu'ils accomplissent leur mission : **invoquer la magie computationnelle, solliciter le jugement d'un modèle**, réclamer la précieuse Toison — **une réponse utile, formatée, souvent elle-même en JSON, et prête à être interprétée ou affichée**.

Laissons champ libre au LLM (4/4)

JASON et JSON, Quête et Requête ...

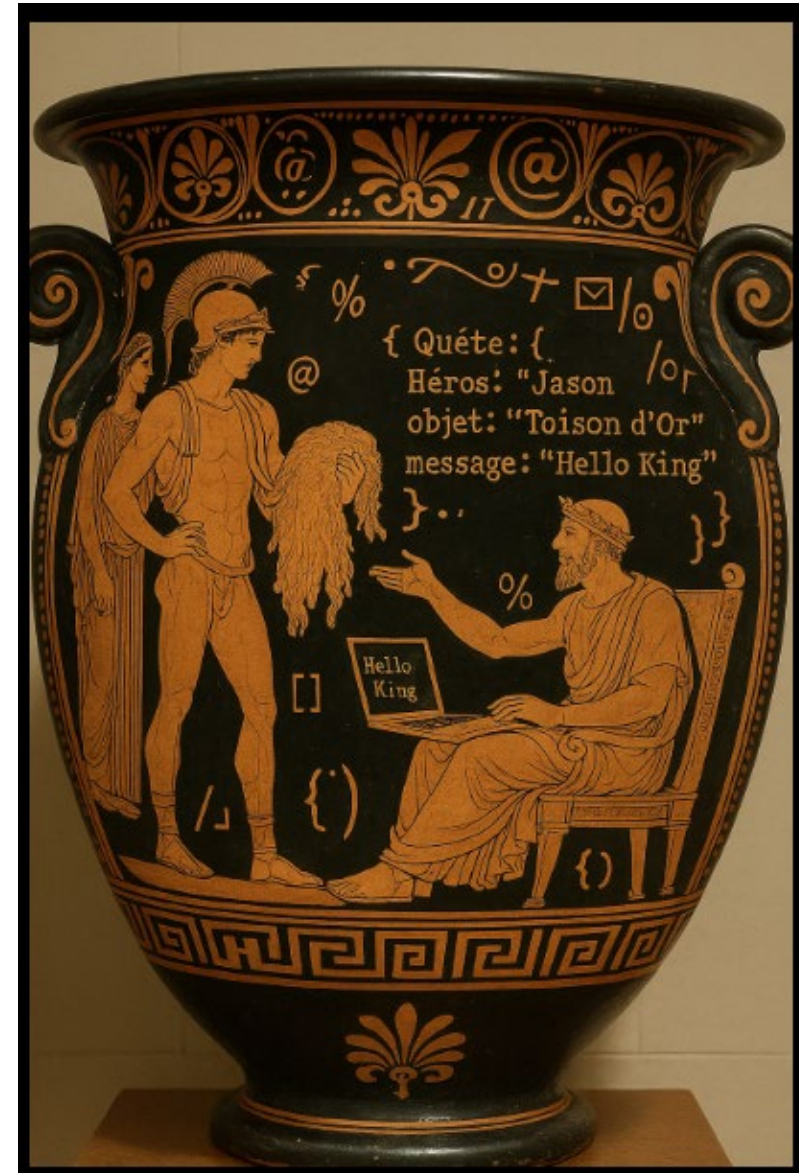
L'IA a répondu :

Et voici deux visuels pour illustrer le récit, typiques des illustrations antiques du 4^{ème} siècle av.JC.

← La requête de JSON (détail)

Le retour de JSON →

Mais peut-être préférerais-tu que nous parlions plus sérieusement du JSON et de son usage dans la communication avec les IA, dans le cadre du test logiciel ?



Plus sérieusement ...

Qu'est-ce que JSON ?

JavaScript Object Notation (JSON) est un **format de données textuel** dérivé de la **notation des objets du langage JavaScript**, largement utilisé pour la **représentation et la transmission d'information structurée**.

Imaginé par Douglas Crockford, le format JSON a été introduit pour la première fois en 2001. Cette démarche a conduit à sa standardisation ultérieure sous la norme ECMA-404 d'Ecma International, publiée en octobre 2003. Il est également décrit en 2017 par la RFC 8259 de l'Internet Engineering Task Force qui se veut compatible avec ECMA-404.

Peut-être avez-vous déjà rencontré JSON en réalisant des **tests d'API** ?

En effet, **la quasi-totalité des API REST modernes utilisent JSON comme format de communication**. Moins verbeux (et plus facile à lire) que XML, moins lourds, JSON permet un **envoi et une réception rapides**, utilisant **moins de bande passante**, ce qui s'avère **utile dans des environnements de test contraints**.

Nativement supporté par presque tous les langages (Python, JavaScript, Java, etc.), les outils de test comme **Postman**, RestAssured, Cypress ou Playwright permettent de **créer facilement des corps de requêtes en JSON** et de faciliter l'interprétation des résultats en **lisant automatiquement les réponses pour faire des assertions sur** des champs JSON (ex. : `pm.response.json().id == 123`).

Plus sérieusement ...

Qu'est-ce que JSON ?

JavaScript Object Notation (JSON) est un **format de données textuel** dérivé de la **notation des objets du langage JavaScript**, largement utilisé pour la **représentation et la transmission d'information structurée**.

Imaginé par [Douglas Crockford](#), le format JSON a été introduit pour la première fois en 2001.

Cette démarche a conduit à sa standardisation ultérieure sous la norme ECMA-404 d'Ecma International, publiée en octobre 2003, et son inscription en 2017 dans la RFC 8259 de l'Internet Engineering Task Force qui se veut compatible avec ECMA-404.

L'objectif de Crockford était de fournir un **format léger, simple et facilement interprétable** pour échanger des données entre un client (navigateur) et un serveur, en particulier dans les applications web.

À la fin des années 1990 et au début des années 2000, JavaScript devenait un langage de plus en plus utilisé côté client (dans les navigateurs) et le besoin d'échanger des données dynamiquement (sans recharger toute la page) devenait crucial.

XML était alors le standard dominant (via SOAP) mais s'avérait très verbeux, complexe à parser côté JavaScript et peu adapté à des interactions rapides et dynamiques.

Douglas Crockford a remarqué que **JavaScript possédait nativement une structure de données idéale : les objets**.

Il a donc extrait un **sous-ensemble syntaxique du langage JavaScript**, parfaitement valide et suffisant pour représenter des structures de données simples.

Plus sérieusement ...

Pourquoi l'IA aime JSON (et les formats structurés)

Au-delà de son usage dans les tests d'API, **JSON** se révèle également particulièrement **utile dans les échanges avec l'IA**.

Les modèles d'intelligence artificielle, en particulier les grands modèles de langage (**LLM**), ont **besoin de deux choses pour bien fonctionner** :

- **Un contexte clair** (des instructions ou une demande compréhensible),
- **Des données bien organisées** (faciles à lire, à interpréter, à transformer).

Plus les données sont ambiguës, désordonnées ou riches en sous-entendus implicites, **plus le risque d'erreur augmente** : hallucinations, incompréhensions, approximations.

Avec des données structurées spécifiées dans le prompt, le raisonnement du LLM sera **cadre par la structure**, et aura moins tendance à en dévier. L'**utilisateur** aura de son côté probablement **mieux réfléchi/structuré** sa demande et son objectif.

Si le prompt **demande expressément** en retour des **données structurées sur le même modèle**, celles-ci seront **plus facilement lisibles/compréhensibles** pour l'utilisateur.

Plus sérieusement ...

Un exemple de réponse JSON

Avant même de détailler le formalisme JSON, examinez le script JSON ci-contre et évaluez le **niveau de clarté et de compréhension** que vous en retirez.

- Que **décrit-il** ?
- Quel **contexte** en déduisez-vous ?
- Imaginez le **prompt préalable**.

```
{
  "titre": "Inscription utilisateur",
  "actif": true,
  "étapes": [
    {
      "action": "Remplir le formulaire",
      "résultat_attendu": "Le bouton 'Valider' devient actif"
    },
    {
      "action": "Cliquer sur 'Valider'",
      "résultat_attendu": "Un message de confirmation s'affiche"
    }
  ],
  "contact": {
    "nom": "MICHU",
    "prénom": "Micheline",
    "email": "mychlayne@yahoo.com",
    "téléphone": null,
    "cp": "92120",
    "ville": "Montrouge"
  }
}
```

Plus sérieusement ...

Un exemple de réponse JSON

Rien de plus facile que de **créer d'autres jeux de données** si la réponse précédente vous convient :

Vous avez dit :

Crée 2 jeux de données passantes
sur ce modèle

Chat-GPT a répondu :

```
[
  {
    "nom": "MICHU",
    "prénom": "Micheline",
    "email": "mychlayne@yahoo.com",
    "cp": "92120",
    "ville": "Montrouge",
    "téléphone": null
  },
  {
    "nom": "Dupont",
    "prénom": "Jean",
    "email": "jean.dupont@example.org",
    "cp": "75015",
    "ville": "Paris",
    "téléphone": "+33 6 12 34 56 78"
  }
]
```

Le cas de test peut aussi être **entièrement rédigé au format JSON**

```
{
  "title": "Inscription utilisateur - saisie des champs obligatoires et facultatif",
  "description": "Vérifier qu'un utilisateur peut valider le formulaire après avoir rempli tous les champs obligatoires ainsi que le champ facultatif.",
  "preconditions": "L'utilisateur est sur la page du formulaire d'inscription. Le formulaire est vierge et affiché correctement.",
  "steps": [
    {
      "action": "Remplir le champ \"Nom\" avec la valeur 'MICHU'»,
      "expected_result": "Le champ 'Nom' contient la valeur saisie"
    },
    {
      "action": "Remplir le champ \"Prénom\" avec la valeur 'Micheline'",
      "expected_result": "Le champ 'Prénom' contient la valeur saisie"
    },
    {
      "action": "Remplir le champ \"Email\" avec la valeur 'mychlayne@yahoo.com'",
      "expected_result": "Le champ 'Email' contient la valeur saisie"
    },
    {
      "action": "Remplir le champ \"Code postal\" avec la valeur '92120'",
      "expected_result": "Le champ 'Code postal' contient la valeur saisie"
    },
    {
      "action": "Remplir le champ \"Ville\" avec la valeur 'Montrouge'",
      "expected_result": "Le champ 'Ville' contient la valeur saisie"
    }
  ],
}
```

SUITE
↓↓↓

Plus sérieusement ...

Un exemple de CT au format JSON (2/3)

```
{
  "action": "Laisser le champ facultatif \"Téléphone\" vide",
  "expected_result": "Aucune erreur n'est affichée"
},
{
  "action": "Vérifier que le bouton 'Valider' est actif",
  "expected_result": "Le bouton 'Valider' est activé"
},
{
  "action": "Cliquer sur le bouton 'Valider'",
  "expected_result": "Un message de confirmation d'inscription s'affiche"
}
],
"priority": "haute",
"tags": ["formulaire", "nominal", "validation"]
}
```

ATTENTION ! N'oubliez pas que l'IA commet des erreurs !

Si vous vous contentez de copier/coller cette réponse dans un outil de test, **un détail risque d'en compromettre l'exécution.**

Saurez-vous le retrouver ?

(Réponse slide suivant)

Réponse : L'erreur est située ligne 7 : » au lieu de ".

```
1{
2  "title": "Inscription utilisateur - saisie des champs obligatoires et facultatif",
3  "description": "Vérifier qu'un utilisateur peut valider le formulaire après avoir rempli tous les champs obligatoires ainsi que le champ facultatif.",
4  "preconditions": "L'utilisateur est sur la page du formulaire d'inscription. Le formulaire est vierge et affiché correctement.",
5  "steps": [
6    {
7      "action": "Remplir le champ \"Nom\" avec la valeur 'MICHU' »,
8      "expected_result": "Le champ 'Nom' contient la valeur saisie"
9    },
10  ],
11}
```

Dans un fichier JSON, remplacer les **guillemets droits (")** par des **guillemets français fermants (»)** ou **ouvrants («)** a des conséquences critiques : **le fichier devient invalide**.

Le fichier sera **inutilisable par les outils d'analyse ou d'échange de données**.

Un parser JSON (dans un langage comme Python, JavaScript, Postman...) renverra une **erreur de syntaxe** (*Unexpected token* ou *Invalid character*).

JSON possède une structure logique simple et efficace.

Nous avons vu que JSON est intolérant aux erreurs de formatage.

Cette apparente rigidité est largement compensée par sa simplicité.

Un document JSON est un **texte délimité**,
qui commence par **{** et se termine par **}**

Il est composé de **valeurs de types composés et scalaires**.

- **Types composés** : Ce sont des structures qui **regroupent plusieurs valeurs**.
Elles permettent de **construire des données organisées**.
Les types composés sont : **Objet** (Object) et **Tableau** (Array)
- **Types scalaires** : Ce sont **des valeurs simples**, non composées.
Les types scalaires sont :
 - **Chaîne de caractères** (String)
 - **Nombres** (Number)
 - **Booléens**
 - **Null**

Ces valeurs doivent répondre à des règles d'écriture strictes.



Objet (Object)

- Il est représenté par des **accolades {}**.
- Il contient des **paires clé/valeur** (appelées aussi "nom/valeur").
- Chaque clé est une **chaîne de caractères entre guillemets**, suivie de **deux-points (:)** puis de la **valeur associée**.

Exemple : `{ "nom": "Alice", "age": 30 }`

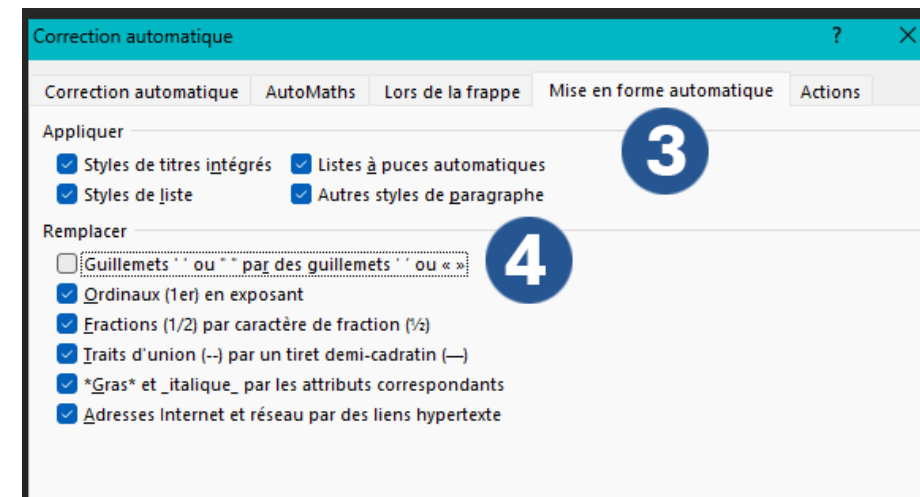
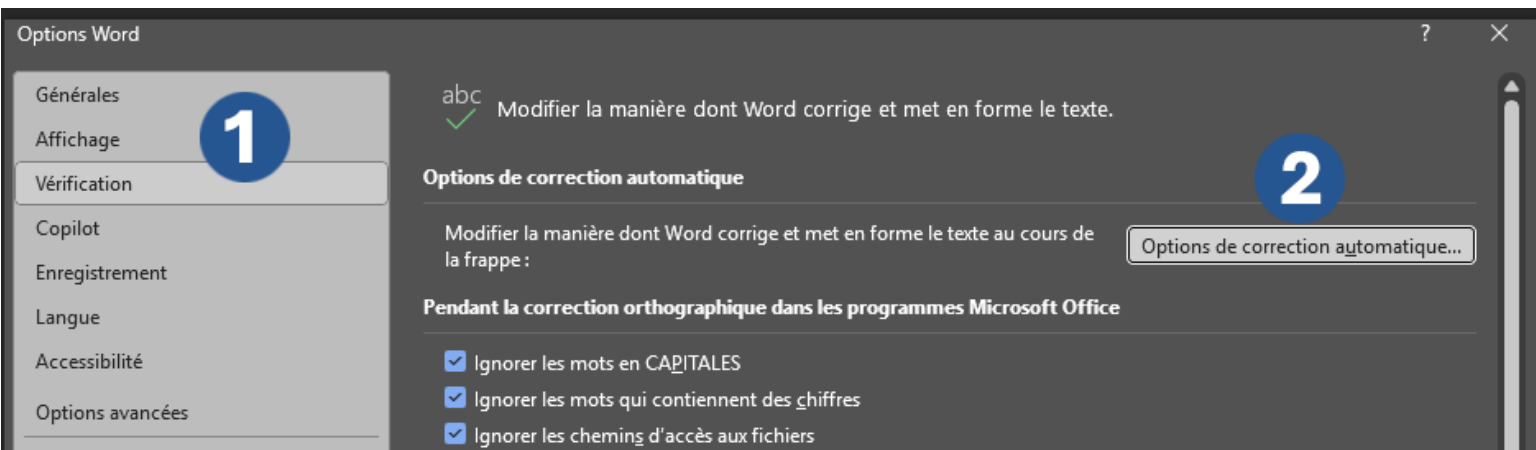
Tableau (Array)

- Il est représenté par des **crochets []**.
- Il contient une **liste ordonnée de valeurs, séparées par des virgules (,)**.
- **L'ordre est important** : chaque valeur est à une **position bien définie (index)**.

Exemple : `[ "pomme", "banane", "cerise" ]`

Chaîne de caractères (String)

- Une suite de caractères Unicode, **entourée de guillemets doubles "**.
- **Unicode** → JSON autorise donc les **caractères accentués** comme é, à, ç, è (avec un encodage UTF-8 !)
- **Attention !** Comme nous l'avons vu précédemment **" ≠ « OU »**
- Vous devrez peut-être, pour afficher correctement **"** notamment dans les logiciels Office, **désactiver l'option de correction automatique des guillemets** : Menu Fichier > Options > Vérification (1) > Options de correction automatique(2) > Mise en forme automatique (3) > Décocher l'option (4)



- **Autre possibilité** : Utilisez Notepad++ > Menu Langages > J > JSON puis copier/coller le texte dans votre document (ou pas, si vous enregistrez directement le fichier au format .json)

Nombre (Number)

- Un nombre peut être **positif** (8) ou **négatif** (-2), **entier** ou **décimal** (-12.5).
- **Attention aux décimaux** : Le point . est le seul séparateur décimal autorisé : **12,5 est de type string !**
- JSON peut utiliser une **notation exponentielle** (ex : $2.5e3 = 2500$).
- **Attention** : JSON n'accepte pas de valeurs comme NaN (Not a Number) ou Infinity.
- JSON ne fait pas la différence entre un **entier/integer** (5) et un **flottant/float** (5.0).

Exemple : { "prix": 12.99, "quantite": 4, "poids": 1.2e2 }



La notation exponentielle des nombres dans JSON

L'écriture **scientifique ou exponentielle** utilisée dans JSON (et en général en informatique) fonctionne ainsi : **XeY signifie : $X \times 10^Y$ (ou 10^Y)**

Exemple : **3.14e2 = 314**

Le chiffre après le **e** (ici 2) indique de combien de décimales on doit décaler vers la droite.

- On a ici $3.14e2 \rightarrow 3.14 \times 10^2 \rightarrow$ on décale la virgule de deux rangs vers la droite $\rightarrow 314$
- Si c'était $3.14e1 \rightarrow 3.14 \times 10^1 \rightarrow$ on décalerait la virgule d'un rang vers la droite $\rightarrow 31,4$
- Si c'était $3.14e-1 \rightarrow 3.14 \times 10^{-1} \rightarrow$ on décalerait la virgule d'un rang vers la gauche (-1 est négatif) $\rightarrow 0.314$

Valeur Null

- **null** signifie **absence de valeur**, ou valeur inconnue.
- Il est souvent utilisé pour indiquer qu'un champ n'a pas encore été renseigné, ou qu'il est facultatif.

Exemple : { "nom": "MICHU", "date_naissance": null }

Booléens

- **Deux valeurs possibles :**
true (vrai)
false (faux)
- **Attention !** Ne pas ajouter de guillemets...
- **Exemple :** { "abonne": true }

BONNE PRATIQUE : Utilisez un validateur JSON pour vérifier la syntaxe de vos fichiers. Par exemple : [JSONLint](#)

- ♦ Très simple à utiliser
- 🔍 Signale clairement les **erreurs de syntaxe**, avec **ligne et colonne**
- 🔄 Option de **reformatage automatique** (pretty print)
- 🌐 Utilisable directement dans un navigateur, sans installation
- 👤 Code open-source disponible : [GitHub – zaach/jsonlint](#)

Un exemple de création de cas de test avec JSON

1. Créer un CT JSON

- Le contexte
- Préparation du prompt (exercice)
- Exemple de réponse (ChatGPT-4o)

Objectif

On souhaite tester les critères d'acceptation d'une User story.

User story à tester :

"En tant qu'utilisateur souhaitant se connecter à l'application mais ayant oublié mon mot de passe,
Je veux que le système m'informe de comment réinitialiser mon mot de passe et m'envoie un mail contenant un lien de réinitialisation

Pour créer un nouveau mot de passe et me connecter à l'application"

Critères d'acceptation :

1. **Given** je suis sur la page de connexion de mon application et j'ai oublié mon mot de passe
When je clique sur le lien "J'ai oublié mon mot de passe"
Then un message est affiché qui m'informe qu'un lien de réinitialisation a été envoyé à mon adresse e-mail
2. **Given** j'ai cliqué sur le lien "J'ai oublié mon mot de passe"
When je me connecte à ma boîte mail
Then un message contenant un lien de réinitialisation est présent dans ma boîte

3. **Given** je clique sur le lien de réinitialisation dans le mail reçu,
When la page de réinitialisation du mot de passe est affichée dans mon application
Then je peux remplir le champ du nouveau mot de passe et le champ de confirmation du nouveau mot de passe
4. **Given** j'ai renseigné le champ du nouveau mot de passe
And j'ai renseigné le champ de confirmation du nouveau mot de passe,
When je clique sur le bouton "Me connecter",
Then je suis connecté à mon compte
And mon tableau de bord est affiché dans l'application

On prévoit d'utiliser le jeu de données suivant :

- **Nom** : MICHU
- **Prénom** : Micheline
- **Adresse e-mail** : michelinemichulastar@caramail.fr
- **Nouveau mot de passe valide** : Mich3l1n3M1CHU!
- **Nouveau mot de passe invalide** : MichelineMICHU@

Le prompt devra comprendre les "6 briques" essentielles :

- rôle assigné au modèle IA,
- contexte clair,
- structure JSON demandée (le JSON Schema),
- données à injecter (jeu de données),
- objectifs précis.

Il devra préciser que la réponse sera un fichier JSON.

➔ Prenez 10 minutes pour ébaucher ce prompt (en binômes)

Exemple de création de cas de test

Prompt généré (1/3)

RÔLE :

Tu es un expert du test logiciel, spécialisé dans les outils de gestion comme Squash TM. Tu conçois des cas de test rigoureux, conformes aux critères d'acceptation définis en amont. Tu respectes scrupuleusement les structures JSON compatibles avec Squash TM.

CONTEXTE :

Je souhaite générer :

- Un cas de test nominal passant, basé sur une séquence complète de réinitialisation de mot de passe.
- Trois cas de test non-passants, pour couvrir 3 erreurs classiques rencontrées pendant ce processus :
 1. Erreur de confirmation du mot de passe
 2. Non-réception du mail de réinitialisation
 3. Utilisation d'un lien expiré ou invalide

Voici la user story et ses critères d'acceptation :

- User Story :

En tant qu'utilisateur souhaitant se connecter à l'application mais ayant oublié mon mot de passe

Je veux que le système m'informe de comment réinitialiser mon mot de passe et m'envoie un mail contenant un lien de réinitialisation

Pour créer un nouveau mot de passe et me connecter à l'application

Exemple de création de cas de test

Prompt généré (2/3)

Critères d'acceptation :

1. Given je suis sur la page de connexion
When je clique sur le lien "J'ai oublié mon mot de passe"
Then un message m'informe qu'un lien de réinitialisation a été envoyé à mon adresse e-mail
2. Given j'ai cliqué sur "J'ai oublié mon mot de passe"
When je me connecte à ma boîte mail
Then un mail contenant un lien de réinitialisation est présent
3. Given je clique sur le lien dans le mail
When la page de réinitialisation s'affiche
Then je peux saisir un nouveau mot de passe et sa confirmation
4. Given j'ai renseigné les deux champs
When je clique sur "Me connecter"
Then je suis connecté à mon compte et mon tableau de bord est affiché

JEU DE DONNÉES À UTILISER :

- Nom : MICHU
- Prénom : Micheline
- Adresse e-mail : michelinemichulastar@caramail.fr
- Nouveau mot de passe (valide) : Mich3l1n3M1CHU!
- Nouveau mot de passe (invalide) : MichelineMICHU@

Exemple de création de cas de test

Prompt généré (3/3)

FORMAT :

Tu dois générer un tableau JSON avec 4 objets, chacun représentant un cas de test.

Chaque objet suit le schéma suivant :

JSON Schema à respecter :

```
{
  "type": "TestCase",
  "name": "Titre du cas de test",
  "objectif": "Finalité pédagogique ou scénario couvert",
  "description": "Description claire du but du test",
  "preconditions": "Contexte de départ pour pouvoir exécuter ce test",
  "importance": "HIGH | MEDIUM | LOW",
  "status": "READY",
  "automation": "MANUAL",
  "nature": "Functional",
  "steps": [
    {
      "action": "Action précise de l'utilisateur",
      "expectedResult": "Résultat attendu (comportement de l'application)"
    }
  ]
}
```

Résultat du prompt / Création des cas de test :

Injectez ce prompt dans un LLM de votre choix, celui-ci fournira les cas de test demandés.

Consultez ce résultat, analysez-le en vous demandant :

- Est-il utilisable "tel quel" ? Sinon, pouvez-vous améliorer/enrichir ces cas de test ?
- Ce résultat est-il "rentable" par rapport à la rédaction manuelle sans assistance ?
- Après cette phase assistée par l'IA, seriez-vous en mesure d'être plus efficace pour créer des cas de test adaptés ?

Il serait possible de s'arrêter là, mais on peut capitaliser sur le travail déjà réalisé !

Par exemple :

Vous avez dit :

Peux-tu me suggérer d'autres améliorations qui amélioreraient la qualité de la réponse et son intégration dans Squash ?

ChatGPT a répondu :

Oui, plusieurs améliorations ciblées peuvent nettement améliorer la qualité de la réponse générée, sa compatibilité avec Squash TM

et sa valeur pédagogique,

Voici un tableau synthétique des axes d'amélioration, suivis d'un détail pour chacun :

Exemple de création de cas de test

Résultat et raffinement du prompt

Axe	Amélioration proposée	Bénéfice concret
 Objectif	Clarifier la finalité du champ "objectif"	Mieux guider l'IA et le testeur sur la portée pédagogique ou fonctionnelle
 Étapes (<i>steps</i>)	Demander explicitement plusieurs étapes avec des enchaînements logiques	Meilleure lisibilité, exécution plus fluide
 Données de test	Intégrer une section "testData" optionnelle	Facilite la reproductibilité et l'import dans Squash si champs dynamiques
 Tags & mots-clés	Ajouter un champ "tags" avec valeurs guidées	Facilite le tri, la recherche et la catégorisation dans Squash
 Résultat attendu global	Ajouter un champ "expectedResult" global au test (en plus des steps)	Vision d'ensemble du comportement attendu en fin de test
 Structuration	Définir une numérotation automatique ou ordonnée des steps	Meilleure lecture dans l'interface de Squash TM
 Groupe de tests	Ajouter un champ "campaign" ou "testSuite" pour rattacher les tests	Intégration facilitée dans les plans de test réels

Souhaites-tu que je reformule ton prompt initial pour intégrer ces améliorations ?

Examinez ces propositions d'amélioration:
Sont-elles toutes pertinentes ? Lesquelles retenez-vous ?

Suivons ses préconisations !

Une amélioration semble peut-être inutile (et nous ne la retiendrons pas) : "Définir une numérotation automatique ou ordonnée des steps", Squash le faisant nativement. On réalise une des limitations du modèle qui "connait" Squash, mais n'a pas de vision pratique de son utilisation ni de ses fonctionnalités. *(On pourrait imaginer, dans le cadre d'une IA "privée", fournir la documentation de Squash et des exemples de projets Squash au modèle pour pallier/limiter ce type d'hallucinations).*

La proposition "Ajouter un champ "campaign" ou "testSuite" pour rattacher les tests" ne semble pas non plus présenter beaucoup d'intérêt, mais ... on verra plus tard.

Continuons la conversation.

Vous avez dit :

OK, intègre toutes ces améliorations sauf le "stepNumber". Ajoute aussi une contrainte pour que les steps de chaque cas de test décrivent toutes les actions à exécuter, depuis le clic sur le lien "J'ai oublié mon mot de passe" sur la page de connexion à l'application.

ChatGPT a répondu :

Parfait, voici la version reformulée et enrichie de ton prompt intégrant toutes les améliorations souhaitées (sauf stepNumber), tout en restant compatible avec Squash TM :

Téléchargez le prompt révisé ([PROMPT RÉINITIALISATION RÉVISÉ.pdf](#)), exécutez-le ou téléchargez le fichier JSON produit ([cas de test reinitialisation final.json](#)) et analysez-le.

L'usage de la structure JSON peut être étendu aux tests exploratoires.

L'IA peut aider les testeurs à mettre en œuvre des tests exploratoires.

Si ce type de test est en général peu structuré, et se fonde sur l'expérience des testeurs, il est recommandé de procéder **en équipe**, dans le cadre de **sessions time-boxées** orientées par une **charte de test**.

Faire appel à un LLM aidera à constituer cette charte de test et organiser la session, tout en étant susceptible de compenser l'éventuel manque d'expérience des testeurs en apportant de nouvelles idées issues de son entraînement.

Astuce : On utilisera la technique de chain-prompting

1. On demandera à l'IA de générer dans une **charte de test** exploratoire une **liste de problèmes/bugs/erreurs** rencontrés dans notre contexte au cours de sa "longue expérience de testeur".

- [Voir le prompt](#)
- [Voir la charte produite](#)

2. Un **deuxième prompt** lui demandera de créer des cas de test en lui **transmettant en entrée la charte de test** précédemment obtenue

- [Voir le prompt](#)
- [Voir la charte produite](#)